

Ex No: 2 Date: 27-02-26	Building and Training a Deep Neural Network for Binary Classification
--	--

Objective:

To build and implement a Deep Neural Network (DNN) classifier from scratch using gradient descent to recognize whether an image contains a car or a bike using the transport dataset.

Descriptions:

Binary classification is the task of classifying elements of a given dataset into two groups. In this project, we classify images into two categories:

1 means a car is present in the image.

0 means a bike is present in the image.

The dataset consists of BMP images stored inside the folders:

transport/cars/

transport/bike/

Each image is resized to $64 \times 64 \times 3$, flattened into a feature vector, and normalized by dividing pixel values by 255. After preprocessing, each image becomes a column vector of size 12288 ($64 \times 64 \times 3$).

Deep Neural Network Model:

Unlike logistic regression (which has no hidden layer) and a 2-layer neural network (which has one hidden layer), this implementation uses a Deep Neural Network with multiple hidden layers.

The architecture used in the code is:

Input Layer $\rightarrow$ Hidden Layer 1 $\rightarrow$ Hidden Layer 2 $\rightarrow$ Hidden Layer 3 $\rightarrow$ Output Layer

Layer dimensions used:

[12288 $\rightarrow$ 20 $\rightarrow$ 7 $\rightarrow$ 5 $\rightarrow$ 1]

Where:

- 12288 is the number of input features (flattened image pixels)
- 20, 7, and 5 are hidden layer neurons
- 1 is the output neuron
-

Activation Functions:

For hidden layers, the ReLU activation function is used:

$$Z[l] = W[l]A[l-1] + b[l]$$

USN NUMBER: 1RUB25CSE0002

NAME: ARUNDATHI B

$$A[l] = \text{ReLU}(Z[l])$$

For the output layer, the Sigmoid activation function is used:

$$Z[L] = W[L]A[L-1] + b[L]$$

$$A[L] = \sigma(Z[L])$$

The sigmoid function produces a probability between 0 and 1.

Cost Function:

Binary cross-entropy loss is used to measure the error between predictions and true labels:

$$\text{Cost} = -(1/m) \sum [Y \log(AL) + (1 - Y) \log(1 - AL)]$$

This cost function is minimized using gradient descent.

Weight Initialization:

Weights are initialized with small random values multiplied by 0.01, and biases are initialized to zero. Random initialization ensures that different neurons learn different features and prevents symmetry problems.

Training Process:

During training:

1. Forward propagation computes predictions (AL).
2. The cost function calculates the error.
3. Backpropagation computes gradients for all layers.
4. Parameters (W and b) are updated using gradient descent.

The update rule is:

$$W[l] = W[l] - \alpha dW[l]$$

$$b[l] = b[l] - \alpha db[l]$$

Where α is the learning rate.

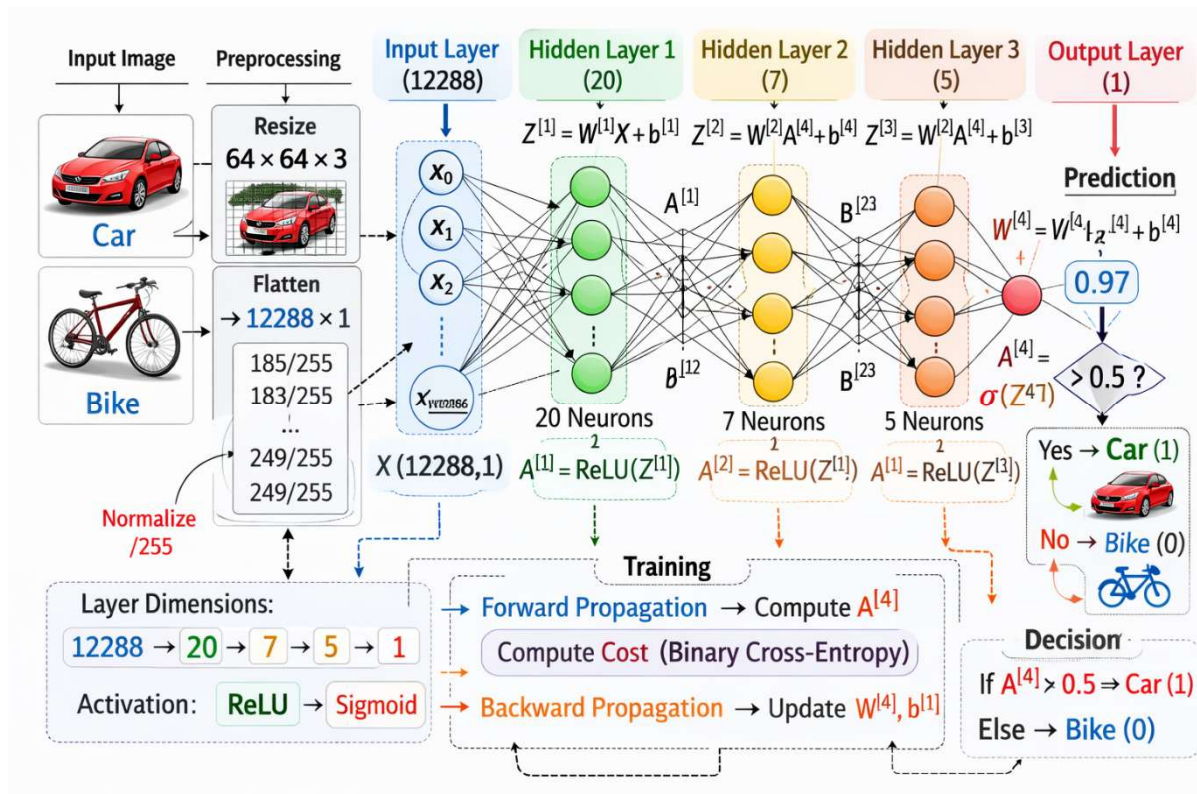
Prediction:

The final output AL represents the probability that the image is a car.

If $AL > 0.5 \rightarrow \text{Car (1)}$

If $AL \leq 0.5 \rightarrow \text{Bike (0)}$

Training accuracy is calculated by comparing predicted labels with actual labels.

Model:**Building the parts of algorithm**

The main steps for building the Deep Neural Network are:

1. Define the model structure (such as number of input features and number of layers)

In this implementation, the model structure is defined as: `layers_dims = [12288, 20, 7, 5, 1]`

Here,

12288 represents the input features ($64 \times 64 \times 3$ flattened image),

20, 7, and 5 represent hidden layer neurons,

and 1 represents the output neuron for binary classification (car or bike).

2. Initialize the model's parameters - The weights are initialized with small random values and the biases are initialized to zeros. This ensures that different neurons learn different features and avoids symmetry problems.

3. Loop:

USN NUMBER: 1RUB25CSE0002

NAME: ARUNDATHI B

- Calculate current loss (forward propagation) - Compute activations layer by layer using ReLU for hidden layers and Sigmoid for the output layer to obtain the final prediction AL.
- Calculate current gradient (backward propagation) - Compute gradients of the cost function with respect to all weights and biases by propagating errors backward through the network.
- Update parameters (gradient descent) - Update all weights and biases using gradient descent to minimize the cost function.

After completing the iterations, the final output is interpreted as:

If $AL > 0.5 \rightarrow \text{Car (1)}$

Else $\rightarrow \text{Bike (0)}$

GitHub Link: https://github.com/karunaarundhati/Deepl_learning_Assignment